Software Technical Proposal

Johan Smith Santamaria Fernández

Sharik Dayanna Rojas Ibarra

Juan Pablo Chala Ramírez

Technologist in Software Analysis and Development

Ficha: 3145556

SENA — Servicio Nacional de Aprendizaje

Neiva, Huila, Colombia, October 2026

Software Technical Proposal

Table of contents

Press F9 (or right-click → Update field) to generate the index.

1 Executive summary

1.1 The proposal in brief

Guardian Escolar is a web and mobile platform for the monitoring and safety of school transportation. It integrates real-time **GPS geolocation**, **QR scanning** of student pickup and drop-off, **automatic notifications and alerts**, a **web administrative panel**, and **mobile applications** for the driver, the guardian, and the student. This technical proposal explains how the platform is built, secured, and launched, with emphasis on the value received by the educational institution and the verifiable delivery conditions.

The proposed model separates the domain into **nine domain services** behind a **single gateway**, with **asynchronous event-driven communication** for real-time operation and a **relational database with a schema per service** to preserve information integrity. The clients are an **Angular 21 web application** with Angular Material 3 and a **React + Expo mobile application**. Delivery is organized into **six phases** and **five verifiable milestones** over **twenty-eight weeks** (preliminary estimate).

1.2 Benefits for the institution

Table 2. Platform benefits for the institution

Benefit	How it materializes
Permanent visibility of the journey	Tracking of the bus position on a map that updates without manual refresh, with a "no signal" marker when the connection is lost.
Certainty about pickup and drop-off	Recording of each boarding through QR scanning, with validation that the student belongs to the route and immediate notification to the guardian.
Timely communication	Automatic notices of pickup, drop-off, prolonged stop, incidents, and loss of signal, directed to the correct recipients.
Centralized operational control	Administration of campuses, courses, people, fleet, routes, schedules, and assignments from a panel with roles and permissions.
Traceability for incidents	History of boardings, route executions, alerts, and administrative actions with a record of who, what, when, IP, and application.
Frictionless adoption	Web experience for administration and mobile experience for field operations, with access differentiated by role.

1.3 Key figures of the proposal

Table 3. Key figures of the proposal

Element	Value
Functional requirements	30 RFs organized into 10 functional groups; 8 non-functional requirements
Stories and epics	20 user stories in 7 epics
Domain services	9 microservices
Data model	38 tables in 11 schemas
System roles	5
Domain events	6 Kafka topics
Client applications	1 Angular 21 web + Angular Material 3; 1 React + Expo mobile
Availability objective	Monthly SLO of 99.9%
Latency objective	API latency P95 < 300 ms at critical points

Element	Value
Estimated duration	28 weeks (preliminary estimate)
Estimated effort	107–156 person·weeks (estimate)

2 Diagnosis and Problem

2.1 Current situation of school transportation

Educational institutions today manage school transportation with largely manual mechanisms and with information scattered among the institution, the driver, and the guardians. There is no single, reliable source that makes it possible to answer, in real time, where the bus is, whether a student picked up or dropped off correctly, and what happened during the journey. That absence of centralized information produces uncertainty among families, hinders the institution's operational control, and eliminates traceability over routes and boardings.

Communication is handled through informal channels —calls, messages, and verbal notices—, which introduces delays, depends on the availability of the people involved, and leaves no record at all. External tracking systems, when they exist, show vehicles on a map, but do not integrate students, guardians, routes, and pickup and drop-off events in a single environment. The result is a fragmented process: either the vehicle position or the class roster is known, but not both elements articulated with the correct person.

2.2 Diagnosis findings

Table 4. Diagnosis findings

#	Finding	Implication
1	There is no single source of school transportation information.	Each actor handles their own version of the facts; coordination and verification are difficult.
2	There is no integrated mechanism to record student pickup and drop-off.	Control is manual, with a risk of human error and no traceability per student.
3	The bus position and the route status are not known in real time.	Guardians depend on third parties to know whether the journey is progressing normally.
4	In the event of incidents or delays, there are no timely alerts or formal channels.	The response to an eventuality occurs late and without a record of what happened.
5	External trackers do not articulate vehicle, student, guardian, and route.	It forces consulting several tools or adapting processes manually.
6	There is no automated record of responsibility per vehicle, route, or action.	The ability to audit assignments and operational decisions is lost.

2.3 Limitations of the current solution

Table 5. Limitations of the current solutions

Current solution	Limitation	Friction it generates
Direct communication between institution, driver, and guardians	Information depends on manual contact and there is no single source of truth.	Additional time and delays in communication.
Manual verification of the journey status	It does not offer a centralized view of the bus's current position.	Uncertainty and dependence on third parties.
Manual control of students during pickup and drop-off	There is no automatic mechanism that records each event per student.	Risk of human error and loss of traceability.
External GPS tracking platforms	They show vehicles, but do not integrate students, guardians, routes, or boardings.	Need to consult different means or adapt processes by hand.

2.4 Evidence collected

Table 6. Evidence collected in the diagnosis

Type of evidence	Key finding
Interviews with users involved in school transportation	The need to know the bus location and to keep guardians informed was identified.
Interviews with those responsible for the service	The need to control who picks up and who drops off the vehicle was identified.
Direct observation of the current process	There is no integrated mechanism to record pickup and drop-off by QR code.
Comparative analysis of tracking solutions	The visualization of vehicles on a map is confirmed as a reference for route tracking.
Technical validation with a GPS device	The feasibility of receiving positioning data over a TCP channel to obtain the vehicle location was confirmed.

2.5 Context constraints

Table 7. Context constraints

Type	Constraint	Implication for the proposal
Academic	Development within the framework of the Software Analysis and Development program, cohort 3145556.	The delivery schedule adjusts to the training period.
Geographic	Initial operating context in Neiva (Huila), Colombia.	Language, data, and operating assumptions are defined for that context.
Technological	Web Angular 21, mobile React + Expo, Java and C# services, SQL Server database.	The design and tests align with that stack.
Development	Docker and containers to standardize execution.	The environments reproduce the same configuration throughout the cycle.

Type	Constraint	Implication for the proposal
Security	Sensitive data of students and families.	Access control, encryption in transit, and least privilege are required.
Regulatory	Colombian personal data protection regulations, with special attention to minors.	Processing, consent, and retention must be explicit.
Delivery	External GPS and notification integrations still without a confirmed provider.	They cannot be considered production-ready until the contract is closed.
Process	Branches develop, main, and docs; commit messages with convention.	Changes and releases follow the documented flow.

3 Proposal Objectives

3.1 General objective

Design, build, and launch a centralized platform that guarantees the traceability and communication of school transportation, so that the institution, drivers, and guardians share the same reliable source of information about each journey.

3.2 Specific objectives

Table 8. Specific objectives and verifiable results

#	Objective	Verifiable result
O1	Guarantee real-time traceability of every school journey.	Bus position available on the map during route execution, with a record of the execution and its events.
O2	Notify guardians of the pickup and drop-off status of their students.	Each boarding recorded by QR generates the corresponding notification to the student's guardian.

#	Objective	Verifiable result
O3	Provide the institution with a panel to manage fleet, routes, and users with roles.	Operational web panel with management of campuses, courses, people, fleet, and routes, restricted by role and permissions.
O4	Incorporate security and data protection from the design stage.	Authentication with a signed token, role-based authorization, auditing of critical actions, and responsible handling of minors' data.
O5	Deliver an operable and observable system.	Defined environments, integration and release pipeline, health checks, metrics, and alerts.
O6	Enable adoption by the five roles.	Interfaces differentiated by role on web and mobile, with verified end-to-end flows.

3.3 Assumptions and external dependencies

Table 9. Assumptions and external dependencies

ID	Dependency	Type	Necessary for	Status	Risk if delayed
D-01	SQL Server instance and credentials	Infrastructure	All data schemas	Assumed available	Data model blockage
D-02	Apache Kafka cluster	Infrastructure	Asynchronous functions	Assumed available	Blockage of events and notifications
D-03	Definition of the route and exception taxonomy	Product	Enumerations and business rules	Open	Incorrect values in types and states
D-04	Push notification provider contracts	External	Notification dispatch	Open	Limited or simulated notifications

ID	Dependency	Type	Necessary for	Status	Risk if delayed
D-05	Initial load of the institution's data	Data	Demonstration and launch	Open	Impossibility of acceptance with real data
D-06	Format of the GPS device stream	External	Positioning ingestion	Open	Live tracking unavailable

4 Solution and Scope

4.1 Solution description

Guardian Escolar is composed of four blocks built on a common identity and data foundation:

- **Services platform**: nine domain services that expose the capabilities of identity, users, school, fleet, routes, notifications, exceptional usages, audit, and settings.
- **Single API Gateway**: entry point that terminates encryption in transit, applies rate limiting and CORS, validates the access token, and routes the versioned requests /api/v1/... to the services.
- **Clients**: Angular 21 web application with Angular Material 3 for administration, and React + Expo mobile application for the driver, guardian, and student, with a camera for QR scanning and push notifications.
- **Data plane**: a relational instance with a **schema per service** (38 tables in 11 schemas), an event broker for the asynchronous flow, and a low-latency channel for positioning telemetry.

4.2 Guiding principles

Table 10. Guiding principles of the solution

Principle	Implication for decisions
Security first	Functions that improve the safety, identification, traceability, or control of transportation take priority over secondary functionality.
Information when it matters	Users receive relevant information at the useful moment, without unnecessary complexity or excessive notifications.
Simple for each user	The platform must be understandable for guardians, drivers, and administrators with different technical levels.
Traceability by default	Important transportation events leave a reliable and consultable record when needed.
Reliable before advanced	GPS tracking, QR recording, routes, and notifications must be reliable before adding complex features.

4.3 Included scope

Table 11. Capabilities included in the scope

#	Included capability	Description
1	Identity and roles	Login, role-based dashboards, and differentiated access for administrators, super-administrators, drivers, guardians, and students.
2	Family and guardian management	Administration of families, guardians, members, and their relationships with students.
3	Student management	Registration of students and association with families, routes, stops, and transportation assignments.

#	Included capability	Description
4	Driver management	Administration of drivers and their license information.
5	Vehicle management	Administration of buses and their operational assignments.
6	Route and stop management	Creation and maintenance of routes, stops, stop order, and assignments.
7	School management	Administration of institutions, campuses, courses, and groups.
8	Route consultation	Consultation of students, routes, stops, and transportation information by guardians and drivers.
9	Monitoring and traceability	Recording of route activity, vehicle status, pickup and drop-off events, and operational history.
10	Notifications and alerts	Communication of delays, detours, incidents, route start and end, and pickup and drop-off events.
11	Profile and data changes	Consultation and updating of email, credentials, and contact data by administrators and super-administrators.
12	Web and mobile access	Angular web experience and React + Expo mobile experience.

4.4 Excluded scope

Table 12. Elements excluded from the scope

#	Excluded element	Reason	Future version?
1	Charges, billing, or settlement of the service	Outside the purpose of safety, monitoring, and management.	Possible future module

#	Excluded element	Reason	Future version?
2	Fleet maintenance, fuel, and mechanical workshop	Does not correspond to a confirmed flow of the current scope.	Possible future version
3	Academic management, grades, or school attendance	Guardian Escolar manages transportation, not the academic record.	No current commitment
4	Public transportation and non-school routes	The domain is limited to school transportation.	No
5	Guaranteed integration with a production GPS provider	The provider and its contract are not confirmed.	Yes, after defining the provider
6	Guaranteed integration with a push notification provider	The production integration is pending confirmation.	Yes
7	Automatic emergency response or dispatch	Alerts communicate incidents; the human response is external.	Requires a separate agreement
8	Biometric identification or facial recognition	Not necessary for the current scope and increases privacy obligations.	To be decided
9	Multi-country regulatory localization	The initial target is Neiva (Huila), Colombia.	Possible future expansion
10	Automatic generation of optimal routes with artificial intelligence	Does not contribute to the traceability objective of the initial scope.	Not in this version
11	Autonomous navigation for the driver	Driving and navigation are outside the system.	No
12	Replacing each institution's emergency procedures	The institutional response is external to the software.	No

4.5 Actors and roles

Table 13. Actors, profile, and main channel

Role	Profile	Main channel
SUPER_ADMIN	Platform operator: manages institutions, global configuration, and audit.	Web
ADMIN	School administrator: administers campuses, courses, people, fleet, routes, and reports of their institution.	Web
DRIVER	Driver: executes routes, confirms boardings with QR, and receives alerts in the mobile application.	Mobile
PARENT	Guardian: follows the route, receives pickup and drop-off notifications, and consults the history.	Mobile
STUDENT	Student: QR identification linked to their profile to board the vehicle.	Mobile

5 Architectural Model and Stack

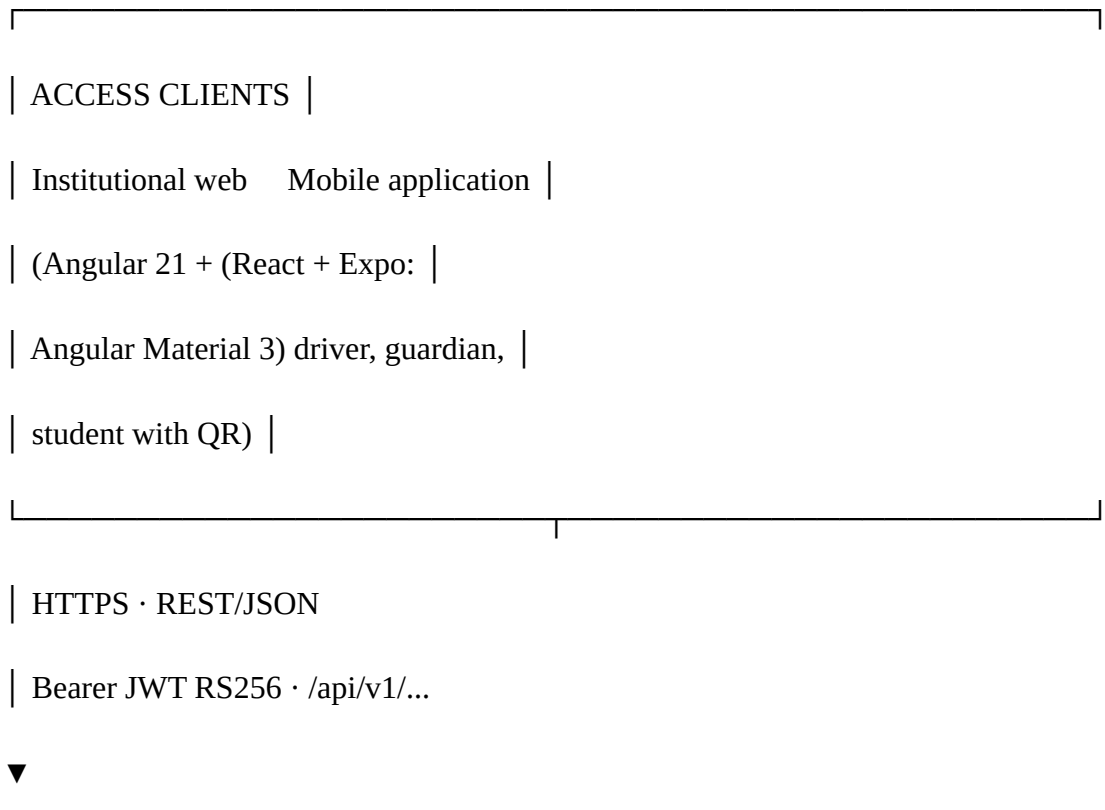
5.1 Architectural style and technical principles

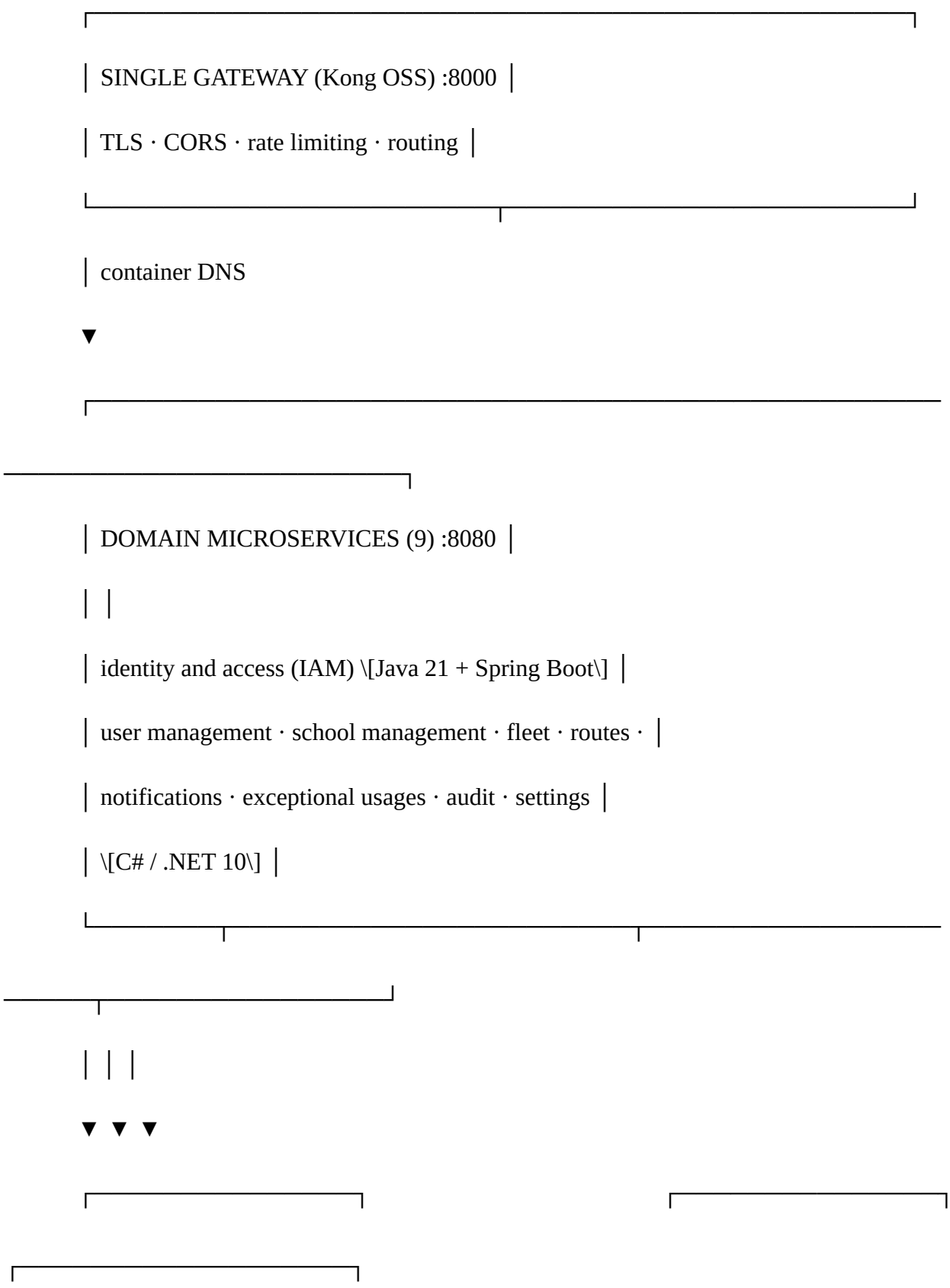
A style of **event-driven microservices** behind a **single gateway** was adopted. The decision responds to the fact that the domain naturally divides into bounded contexts (identity, school, transportation, tracking, notification, and infrastructure), to the fact that real-time flows —boardings, positioning, and alerts— are better processed asynchronously, and to the fact that each service can evolve and be deployed without stopping the rest. The database is a **single relational database with a schema per service**, which preserves referential integrity and reduces operating cost compared with one instance per service.

Table 14. Technical principles of the architectural style

Principle	Application
Contracts first	Interface contracts are defined before implementation and are consumed from a documentation portal over the OpenAPI standard.
Schema per service	Logical isolation per service within the same relational instance; no mixing of data responsibilities.
Fail fast, recover gracefully	Validation at the edge, health checks, circuit breakers on synchronous calls, and idempotent consumers.
Observability by design	End-to-end correlation identifier, structured logs, metrics, and traces per service.
Security by default	Signed token at the edge, role-based authorization with deny by default, secrets outside version control, and auditing of sensitive actions.

5.2 General view of the proposed model




SINGLE GATEWAY (Kong OSS) :8000
TLS · CORS · rate limiting · routing
container DNS
DOMAIN MICROSERVICES (9) :8080
identity and access (IAM) \[Java 21 + Spring Boot\]
user management · school management · fleet · routes ·
notifications · exceptional usages · audit · settings
\[C# / .NET 10\]

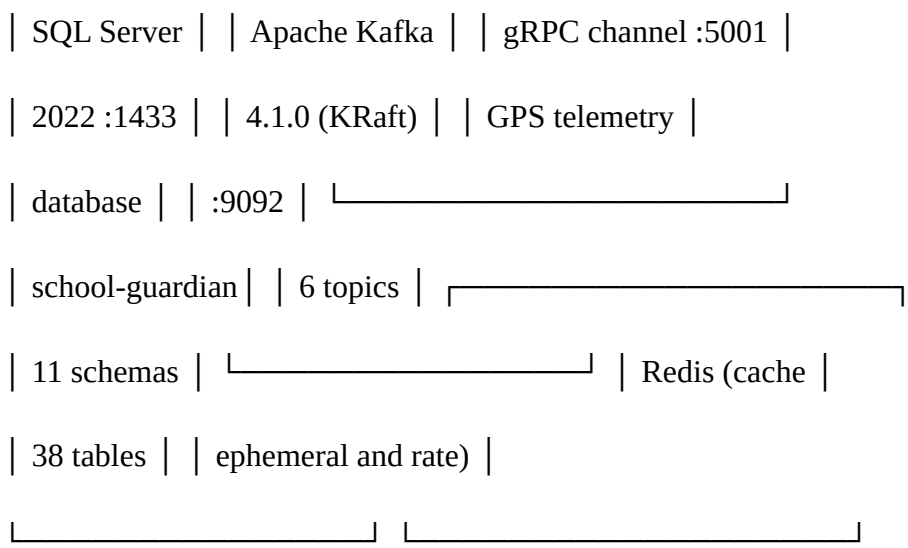


Figure 1. General view of the proposed architecture

5.3 Components and ports

Table 15. Components, technology, and ports

Component	Main function	Technology	Port
API Gateway	Single entry point: routing, TLS, CORS, rate limiting	Kong OSS	8000
Identity and access service (IAM)	Authentication, roles, permissions, profiles, and sessions	Java 21 + Spring Boot	8080
User management service	People, families, family links, and driving licenses	C# / .NET 10	8080
School management service	Institutions, campuses, courses, and cities	C# / .NET 10	8080
Fleet service	Buses, brands, models, and driver–bus assignments	C# / .NET 10	8080
Route service	Routes, stops, schedules, assignments, and executions	C# / .NET 10	8080

Component	Main function	Technology	Port
Notification service	Boardings, alerts, recipients, and dispatch	C# / .NET 10	8080
Exceptional usage service	Use of the bus by third parties and route detours with a reason	C# / .NET 10	8080
Audit service	Recording of critical actions and errors	C# / .NET 10	8080
Settings service	Management of the password policy and security settings	C# / .NET 10	8080
Event Broker	Asynchronous decoupling between services	Apache Kafka 4.1.0 (KRaft)	9092
Relational storage	Persistence with schema per service	SQL Server 2022	1433
Telemetry channel	Low-latency positioning	gRPC	5001
Ephemeral cache	Session cache and rate control	Redis	6379

5.4 Communication patterns

Synchronous client–platform communication uses **REST/JSON** over the gateway, with /api/v1 versioning. The services exchange **events** through the broker for the high-concurrency and asynchronous flows, and use **gRPC** on a persistent channel for the low-latency positioning telemetry. Responses maintain a consistent error structure (timestamp, message, details/path) and pagination by page and size.

5.4.1 Domain events

Table 16. Domain events and their participants

Topic	Purpose	Functional producer	Functional consumer
student.created	Student registration	User management service	Identity and access service
driver.created	Driver registration	User management service	Identity and access service
admin.created	Administrator registration	User management service	Identity and access service
parent.created	Guardian registration	User management service	Identity and access service
profile.created	Profile created in identity	Identity and access service	Subscription in preparation
student.scanned	QR scan for pickup or drop-off	Notification service (scan)	Notification service (boarding and dispatch)

5.4.2 Flow of a boarding

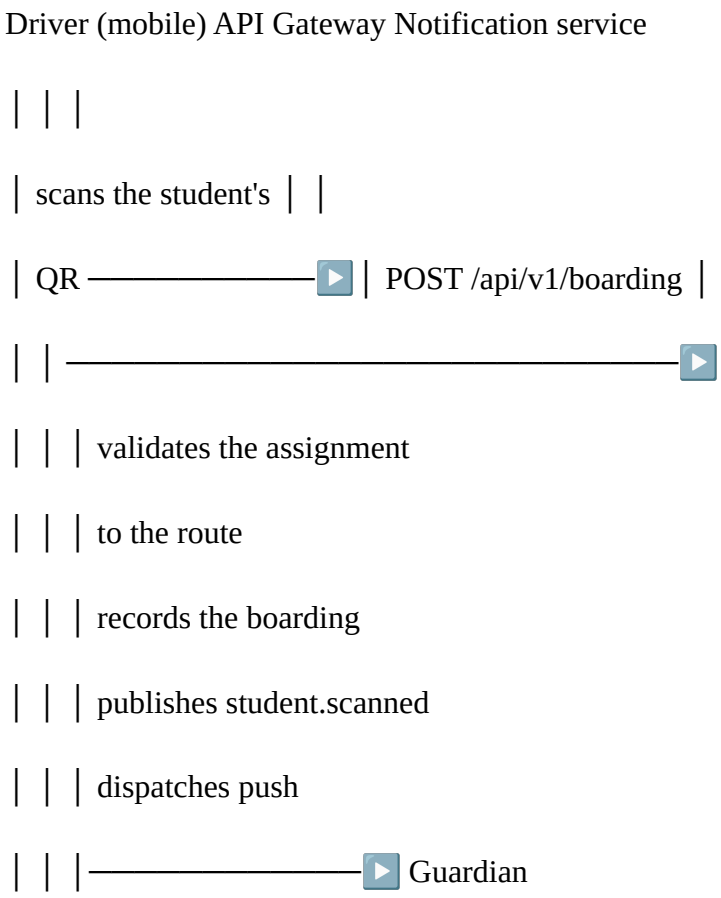




Figure 2. Flow of a boarding by QR scanning

5.4.3 Representative resources

Table 17. Representative resources of the interface

Method	Path	Role	Description
POST	/api/v1/auth/login	Public	Login and issuance of the access token
POST	/api/v1/boarding	DRIVER	Registration of a boarding by QR scanning
GET	/api/v1/buses/{busId}/ location	PARENT, ADMIN	Bus position for the tracking map

The interfaces are organized by functional domain: authentication and sessions; people and profiles; families; institutions and campuses; courses; fleet; routes; geolocation; notifications, alerts, and boardings; exceptional usages; settings; and audit.

5.5 Data model summary

The model consists of **38 tables distributed across 11 schemas**, with UUID primary keys (except auto-increment catalogs and audit tables), declared foreign keys, datetime2 dates, and logical deletion through Status with ACTIVE/INACTIVE states.

Table 18. Content of the data schemas

Schema	Key content
Iam	Profile (person, campus, hash, and role), Role, Permissions, PermissionsRole, SessionProfile
UserManagement	Person, Family, FamilyMember (guardian/student relationship), DriverLicense

Schema	Key content
School	School, SchoolCampus, Course, SchoolAdmin
Geographic	City
Fleet	Bus, Brand, Model, DriverAssignemts
Route	Route, Stop, RouteStop, RouteBusAssignments, RouteStudentAssignments, RouteSchedule, RouteExecution
Gps	GpsDevice, GpsLocation
Notification	Boarding, Alert, AlertType, AlertRecipient, DeviceToken
Exceptional	ExceptionalDriverUsage, ExceptionalRouteUsage
Settings	PasswordPolicies, SecuritySettings
Audit	Audit, LogErrors

The central business relationships are institution → campus → (bus, route, profile), route → ordered stops, schedule → bus and route assignments, execution → boardings and alerts, and person → profile → (role, family link, license, session, audit).

5.6 Justification of the stack by layer

5.6.1 Services layer

Table 19. Services layer decisions

Decision	Selection criterion	Alternatives considered
Event-driven microservices	The domain divides into bounded contexts; it allows deploying each capability independently and processing boardings, position, and alerts asynchronously.	Modular monolith; single layered architecture
Kong OSS API Gateway	Single entry point with TLS, rate limiting, CORS, and versioned routing, without tying the system to a language.	Nginx with custom logic; gateway included in the framework; commercial product

Decision	Selection criterion	Alternatives considered
Identity service in Java 21 + Spring Boot (Maven)	Mature ecosystem for security, hexagonal architecture, and asymmetric signature libraries.	C# for all services; Node.js
Domain services in C# / .NET 10	Performance, productivity, and homogeneity across most of the services, with container deployment.	Java for all; Go; Node.js
Synchronous REST/JSON communication	Open, versioned contracts that are easy to consume from web and mobile.	GraphQL; RPC over HTTP for all traffic
Low-latency gRPC communication	Persistent and efficient channel for the telemetry and geolocation flow (port 5001).	REST with polling; WebSocket
Messaging with Apache Kafka 4.1.0 in KRaft mode	Ordering by key, event retention, and decoupling, without an external coordination component.	RabbitMQ; queue over the database
Containers with Docker / Docker Compose	Reproducibility of the environments and custom images per service.	Direct installation on the host

5.6.2 Data layer

Table 20. Data layer decisions

Decision	Selection criterion	Alternatives considered
SQL Server 2022, single instance with schema per service	Relational integrity, 38 tables in 11 schemas, lower operating cost, and a documented evolution path toward isolation.	Database per service; NoSQL store
Migrations with Liquibase 4.29	Versioned changesets with deterministic order, valid for both Java and .NET services.	Framework ORM migrations; Flyway
Ephemeral cache in Redis	Volatile state outside the services: session cache and rate counters at the edge.	Gateway memory; database

Decision	Selection criterion	Alternatives considered
Additively versioned events	Prevent schema evolution from breaking consumers.	In-place contract changes

5.6.3 Clients and operation

Table 21. Clients and operation decisions

Decision	Selection criterion	Alternatives considered
Web with Angular 21 + Angular Material 3	Administration components, tables with pagination, validated forms, dashboards with indicators, and role-based navigation.	Vue; React for web; custom HTML and CSS
Mobile with React + Expo	A single codebase for Android and iOS, a camera for QR scanning, and push notifications.	Native development per platform; React Native without Expo
Identification with JWT RS256 (access 15 min / refresh 30 days)	Asymmetric signature: only the identity service holds the private key and the gateway validates with the public key.	Server sessions; token signed with a shared secret
Container deployment with restart and rollback	Fast recovery from failures and releases without interrupting traffic.	Direct publication on the host
Pipeline per service	Early detection of regressions and repeatable releases.	Manual publication
Correlated logs, health, and metrics	Diagnosis between services and verification of the availability objective.	Plain-text logs without correlation

6 Key Functional Capabilities

6.1 Requirements structure

The functional scope comprises **30 functional requirements distributed across 10 groups**, complemented by **8 non-functional requirements** that set quality, security, and

operation.

Table 22. Functional requirements by group

Group	Title	No. of RF
RF-01	User management	4
RF-02	Authentication	2
RF-03	School route management	8
RF-04	QR scanning management (pickup and drop-off control)	3
RF-05	Boarding notifications	4
RF-06	Real-time bus tracking	3
RF-07	Trip management	2
RF-08	Incidents	2
RF-09	Route consultation	2
RF-10	Personalization	2
Total	**10 groups**	**30**

6.2 Capabilities by group

Table 23. Capabilities by requirements group

Group	Main capability	Roles	Channel
RF-01	Registration, editing, and logical deletion of students, drivers, guardians, and administrators; linking students to the guardian's account.	ADMIN, SUPER_ADMIN	Web
RF-02	Login with credentials and token issuance, logout, and access renewal.	All	Web and mobile

Group	Main capability	Roles	Channel
RF-03	Routes with ordered stops and schedules, uniqueness validation, assignment of bus to route, driver to bus, and students to route with their stop.	ADMIN	Web
RF-04	Generation of the student's individual QR code, reading to record the pickup (validating the route assignment) and the drop-off (requiring an active pickup).	DRIVER, STUDENT	Mobile
RF-05	Detection of the pickup and drop-off event, real-time notification to the guardian with time and location, and alert for a prolonged stop.	PARENT	Mobile
RF-06	Continuous capture of the bus position, visualization on a map without manual refresh, and alert for loss of signal with the last known position.	PARENT, ADMIN	Web and mobile
RF-07	Trip start (a single active trip per bus) and trip end validating that no pending drop-offs remain.	DRIVER	Mobile

Group	Main capability	Roles	Channel
RF-08	Incident report with type, optional description, and position, and automatic notification to the institution and to the guardians of the students on board.	DRIVER	Mobile
RF-09	Consultation of the student's route detail by the guardian and of the assigned route by the driver, with bus, driver, ordered stops, and schedules.	PARENT, DRIVER	Web and mobile
RF-10	Configurable color theme and interface in at least four languages.	All	Web and mobile

6.3 Epics and user stories

Table 24. Epics and user stories

Epic	Title	Stories	No.
EP-001	Pickup and drop-off registration by QR	HU-QR-001, HU-QR-002, HU-QR-004	3
EP-002	Notifications	HU-NOTIF-001, HU-NOTIF-005	2
EP-003	Route and fleet management	HU-ROUTE-001, HU-ROUTE-002, HU-ROUTE-003, HU-ROUTE-004, HU-ROUTE-005, HU-ROUTE-008, HU-FLEET-001, HU-BUS-002	8

Epic	Title	Stories	No.
EP-004	User and family management	HU-IAM-001, HU-USER-003, HU-USER-004	3
EP-005	Live tracking	HU-GPS-002	1
EP-006	Incidents	HU-INCIDENT-001	1
EP-007	Interface and personalization	HU-UI-001, HU-UI-002	2
Total	**7 epics**		**20**

Stories are prioritized with a security-first approach: the critical chain —identity, users, routes, fleet, QR scanning, and notifications— is delivered complete before incorporating maturity features such as incident reporting and personalization.

7 Security and Compliance

7.1 Identification, authorization, and sessions

Table 25. Identification, authorization, and sessions

Element	Definition
Authentication	JWT token signed with RS256: **15-minute** access and **30-day** renewal. Only the identity service holds the private key; the gateway validates with the public key.
Public endpoints	Only login and renewal; the rest require the Authorization: Bearer <token> header.
Authorization	Role and permission control through the Permissions and PermissionsRole tables, with deny by default and scope by institution and campus.
Sessions	Record in SessionProfile with IP address and dates, which makes it possible to audit access and revoke sessions.

Element	Definition
Credential policies	Configurable parameters for length, uppercase letters, numbers, symbols, and expiration in days, managed from PasswordPolicies and SecuritySettings.
Account registration	There is no self-registration: accounts are created by the administrator, which reduces the risk of unauthorized access.

7.2 Technical controls

Table 26. Technical security controls

Control	Description	Layer
Encryption in transit and CORS	TLS termination and restriction of the web origin at the gateway; HTTPS mandatory in production.	API Gateway
Rate limiting	Counters in the ephemeral cache to prevent abuse and overloads.	API Gateway
Input validation	Validation at the gateway and in each service before processing the request.	Edge and services
Credential protection	Password hashing in accordance with the policy for length, complexity, and expiration in days.	Identity service
Least privilege	Permissions by role and deny by default; database credentials per schema.	Services and data
Secrets outside version control	Use of environment variables with placeholders such as <JWT_PRIVATE_KEY> and <ADMIN_PASSWORD>.	All environments

Control	Description	Layer
Auditing of critical actions	Recording of who, what, when, IP address, and application in the Audit table, with no update or deletion from the application.	Audit service
Error logging	Persistence of type, description, IP, and date in LogErrors.	Audit service
Protected internal communication	Service channels with network trust and tokens; topics with topic-level access control.	Services and broker
Logical deletion	ACTIVE/INACTIVE states without physical deletion, preserving historical traceability.	Data

7.3 Identified threats and mitigation

Table 27. Identified threats and their mitigation

Category	Threat	Mitigation
Spoofing	Use of a forged token to invoke internal services.	RS256 signature at the edge, rejection of unsigned algorithms, and key rotation.
Tampering	Replay or modification of payloads on the gRPC channel or in the events.	Encryption in transit, topic-level access control, and versioned event schema.
Repudiation	Lack of evidence about who changed an assignment or a route.	Insert-only audit record in the audit schema.
Information disclosure	Code injection in queries or leakage of personal data.	Parameterized queries, database roles with least privilege, and controlled migrations.
Denial of service	Saturation of health or authentication endpoints.	Rate limiting at the gateway and progressive backoff on internal calls.
Privilege escalation	A user's access to another institution's data.	Authorization at the edge and in the service, scope by institution, and boundary rules between services.

7.4 Personal data protection and compliance

Table 28. Handling of personal data protection

Aspect	Handling in the proposal
Applicable framework	Colombian personal data protection regulations, with special attention to the data of minors.
Minimization	Events and payloads between services are limited to the necessary data and do not include secrets.
Consent and authorization	The processing of student data requires authorization from guardians or representatives, managed by the institution.
Masking	Personal data is masked in the logs and is not exposed in error messages or traces.
Retention	The persistence of positioning data must be defined with the legal area before go-live.
Access	Access to personal information is restricted by role and by institutional scope, and is recorded in the audit.

7.5 Audit and traceability

The audit service centralizes the recording of critical actions and errors in an insert-only schema. This evidence is joined by the boarding history (Boarding), the route executions (RouteExecution), the alerts and their recipients (Alert, AlertRecipient), and the sessions (SessionProfile). Together, they make it possible to reconstruct what happened on a journey, who executed an administrative action, and which users were notified.

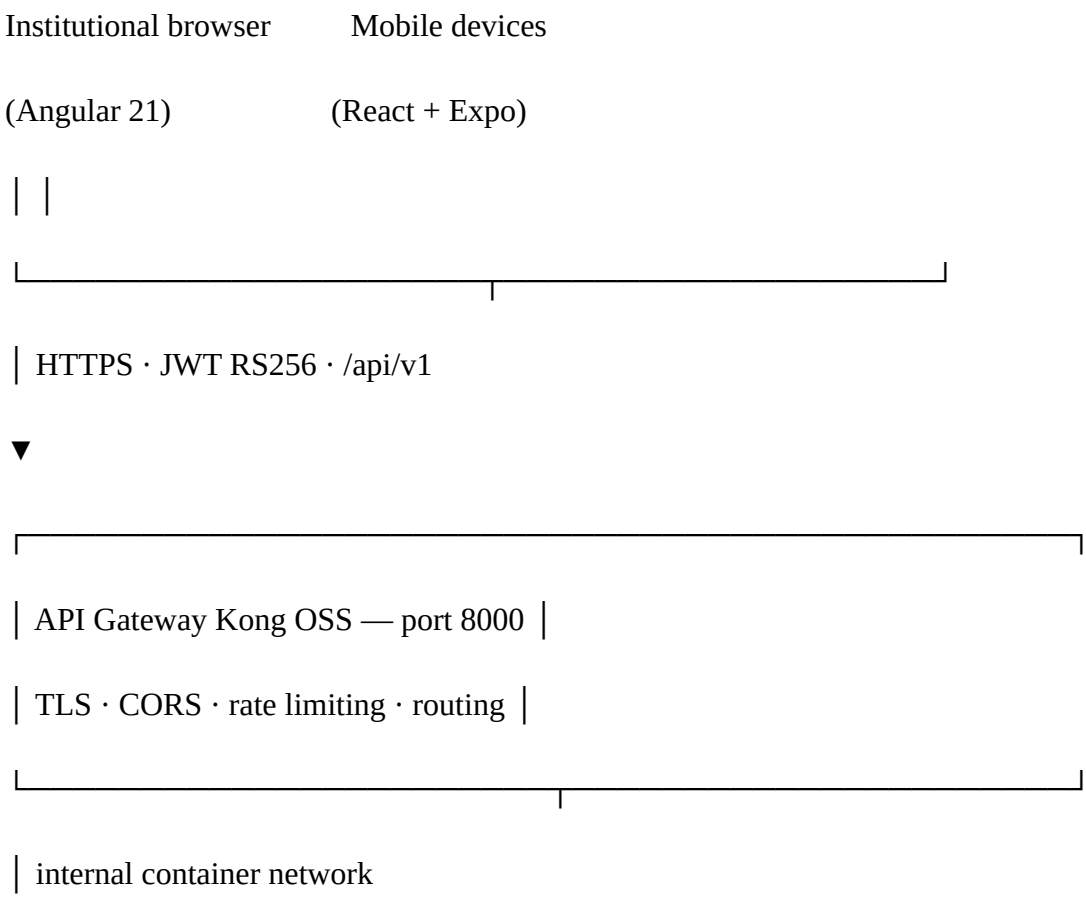
8 Deployment Model and Environments

8.1 Environments

Table 29. Environments, purpose, and update

Environment	Purpose	Update	Access	Data
Local	Development on each member's machine	Manual	Developer	Synthetic or seed
Development	Continuous integration	Automatic on merging a change into the development branch	The whole team	Synthetic
Testing and acceptance	Pre-production, validation, and demonstrations	Automatic on merging into the main branch	Team and institution	Anonymized
Production	Real users	Manual with approval	Technical manager and operations	Real

8.2 Deployment Topology



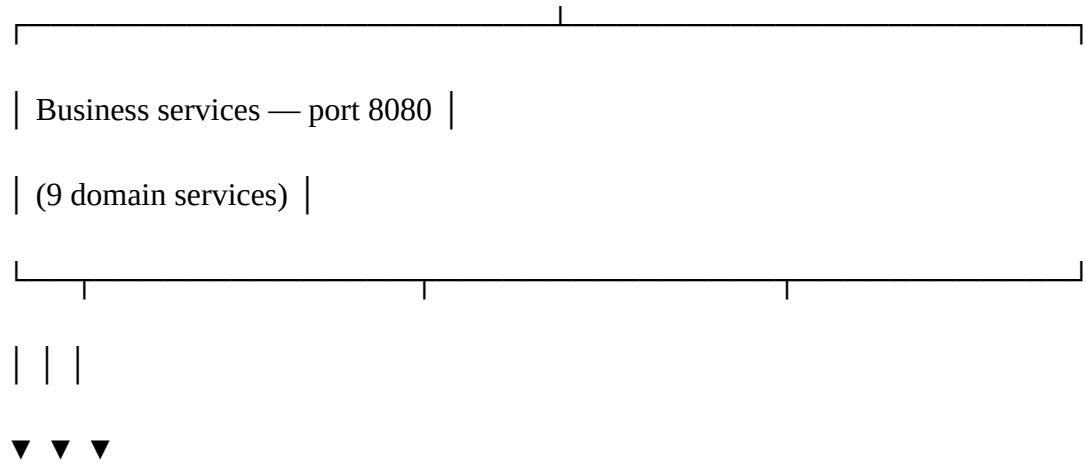


Figure 3. Deployment topology of the platform

8.3 Continuous integration and release

change branch ──▶️ development ──▶️ testing ──▶️ production

format, tests integration, suite canary

unit tests and image and complete, at 10%,

compilation deployment contract, observation

(blocks the with smoke load test (k6) and step to

merge) test and notice to 100% or

the institution rollback

Figure 4. Continuous integration and release flow

Table 30. Pipeline stages and duration

Pipeline	Trigger	Stages	Target duration
Change	Each merge request	Format and static analysis; unit tests; compilation	Less than 10 minutes in total; its failure blocks the merge
Development	Merge into the development branch	Compilation, tests, and analysis; integration tests; image build; deployment; smoke tests	Less than 38 minutes
Testing	Merge into the main branch	Full test suite; contract tests; image build and tagging; deployment; smoke tests; load tests with k6; notice for acceptance	Less than 55 minutes
Production	Manual approval	Canary at 10% of traffic; 15-minute observation; step to 100% or automatic rollback	According to the release window

The automatic controls by type of change include the validation of the documentation and the links, the validation and comparison of API contracts, the compilation of the services, the validation of the migration changesets, and the deployment to development with a smoke test on the gateway.

8.4 Startup Order and verification

1. Start the database system.
2. Start the event broker.
3. Apply the versioned migrations with Liquibase.
4. Deploy the domain services and wait for their health checks.
5. Reload the gateway routes when they change.
6. Publish the web application and distribute the mobile application.
7. Verify availability with the public health check.

curl -fsS <http://localhost:8000/health>

8.5 Release and rollback strategy

Production release uses canary deployment with automatic rollback. Before releasing, the review and approval of the change is verified, along with the passing of the test suite in the testing environment —including performance with k6—, the institution's acceptance approval, the execution of migrations before the code, and the availability of the rollback procedure. In the event of adverse behavior, it reverts to the previous version and, if there were an incompatible migration, the contingency procedure is activated with the approval of the technical manager.

9 Implementation Plan by phases

9.1 Delivery approach

Execution adopts a hybrid approach with short iterations and formal milestone control. The work is organized on a backlog of **20 user stories in 7 epics**, with a tracking board, a

definition of done per increment, and periodic validation with the institution. Each increment relies on the described integration and release pipeline, so that quality and security are verified continuously and not at the end.

9.2 Phases and activities

Table 31. Phases, activities, and results

Phase	Duration (weeks)	Main activities	Result
F1. Analysis and validation	1–2	Confirm the scope, close open questions, define the route and exception taxonomy, validate the initial data load.	Scope and baseline plan approved
F2. Design	3–5	Architecture, data model, interface contracts, screen design by role, environments, and pipeline.	Base environment operational with gateway and database
F3. Iterative construction	6–17	Identity and users; routes, fleet, and assignments; trip operation with QR; tracking and notifications; incidents and personalization.	Functional increments per epic
F4. Testing and integration	18–21	Unit, integration, end-to-end, load, and security tests; defect correction; acceptance with the institution.	Test suite passed and acceptance

Phase	Duration (weeks)	Main activities	Result
F5. Deployment and implementation	22–23	Migrations, deployment per environment, initial data load, training, and distribution of the mobile application.	System in production
F6. Support and stabilization	24–28	Monitoring of the availability objective, adjustments, operation and backup procedures, review of results.	Go-live package and closure

The durations correspond to a preliminary estimate and adjust to the closure of external dependencies and the institution's feedback.

9.3 Milestones and exit criteria

Table 32. Milestones and exit criteria

Milestone	Target week	Exit criterion
H1. Base environment operational	5	Routes configured on the gateway and a satisfactory smoke test on port 8000.
H2. Accounts and routes operational	10	Login, account management, routes, and fleet with their assignments available and verified.
H3. Trip operation with QR	14	Recording of pickup and drop-off by QR scanning and start and end of trip in the mobile application.
H4. Guardian visibility	17	Map with the bus position and pickup and drop-off notifications delivered to the guardian.

Milestone	Target week	Exit criterion
H5. Go-live	23–28	Institution acceptance, production deployment, operation and backup procedures, and results reviewed against the key results.

9.4 Schedule

Weeks 1 5 9 13 17 21 25 28

----	----	----	----	----	----	----

F1 Analysis ##

F2 Design ###

F3 Constr. ############

F4 Testing ####

F5 Deployment ##

F6 Support #####

Milestone H1 H2 H3 H4 H5

Figure 5. Schedule of phases and milestones in weeks

Scale: one column equals one week. The durations and dates are a preliminary estimate subject to confirmation of external dependencies.

10 Resources

10.1 Team and roles

Table 33. Team, responsibilities, and dedication

Role	Main responsibility	Estimated weekly dedication
Technical lead and architect	Architecture decisions, contract review, and code quality.	1.0 person
Service development (Java and .NET)	Implementation of the domain services, events, and API contracts.	1.5–2.0 people
Web application development (Angular)	Screens by role, tables, forms, and dashboards.	0.7–1.0 people
Mobile application development (React + Expo)	Driver, guardian, and student applications; QR camera and push notifications.	0.7–1.0 people
Quality and testing	Test strategy, cases per requirement, automation, and defect management.	0.5–0.8 people
Operations and deployment	Environments, pipeline, migrations, monitoring, and backups.	0.3–0.5 people
Interface and experience design	Screens by role, design system, and user validation.	0.2–0.3 people
Project management and acceptance	Backlog, plan, risks, and acceptance with the institution.	0.2–0.3 people

The dedication corresponds to the construction phase, in which the team reaches its largest size. In the analysis, design, testing, deployment, and support phases the team is reduced. The core project team is made up of three developers from the Software Analysis and Development program, so some functions are covered with partial dedication.

10.2 Required competencies

Table 34. Required competencies by area

Area	Competencies
Services	Java 21 with Spring Boot, C# with .NET 10, REST API design, events, and low-latency RPC.

Area	Competencies
Data	SQL Server, relational modeling, versioned migrations, and efficient queries.
Clients	Angular 21 with Angular Material 3, React with Expo, accessibility, and responsive design.
Quality	Unit, integration, end-to-end, load, and security tests.
Operations	Containers, continuous integration and release, monitoring, and incident response.
Security	Authentication with asymmetric signature, role-based authorization, and personal data protection.

10.3 Infrastructure and tools

Table 35. Infrastructure and tools

Resource	Specification	Use
Container platform	Docker 24.0 or higher and Docker Compose 2.20 or higher	Standardize execution in all environments
Version control	Git with develop, main, and docs branches, and commit messages with convention	Traceability of changes and releases
Database	SQL Server 2022, school-guardian database, port 1433	Persistence of the 11 schemas
Event Broker	Apache Kafka 4.1.0 in KRaft mode, port 9092	Asynchronous flow of domain events
API Gateway	Kong OSS, port 8000	Single entry, security, and routing
Telemetry channel	gRPC, port 5001	Low-latency positioning
Ephemeral cache	Redis, port 6379	Session cache and rate limiting
Migrations	Liquibase 4.29	Versioned changesets per service
Performance tests	k6 in the testing environment	Latency verification before release
Monitoring	Health checks, structured logs with a correlation identifier, metrics, and alerts	Observability of testing and production

11 Effort Estimation by Phase

11.1 Basis of the estimate

The estimate is expressed in **person·weeks** and is presented as a **range**, not as a commitment. It was built by breaking down each phase by activity and applying a variation margin according to the maturity of the external dependencies. It does not include monetary figures, third-party licenses, or provider costs, which must be quoted separately. The values assume a scope of 30 functional requirements and 20 user stories, and adjust if the scope, the number of institutions, or the confirmation of the external providers changes.

11.2 Effort by phase

Table 36. Estimated effort by phase

Phase	Duration (weeks)	Estimated effort (person·weeks)	Key assumptions
F1. Analysis and validation	2	6–8	Small team of 3 to 4 people; closure of open questions with the institution.
F2. Design	3	9–15	Team of 3 to 5 people; architecture, data, contracts, and screens.
F3. Iterative construction	12	60–84	Team of 5 to 7 people on average; construction by epics.
F4. Testing and integration	4	16–24	Team of 4 to 6 people with support from the development team.
F5. Deployment and implementation	2	6–10	Team of 3 to 5 people; migrations, data load, and training.

Phase	Duration (weeks)	Estimated effort (person·weeks)	Key assumptions
F6. Support and stabilization	5	10–15	Team of 2 to 3 people with partial dedication.
Total	**28**	**107–156**	**Preliminary estimate**

11.3 Distribution by role

Table 37. Distribution of effort by role

Role	Person·weeks (estimate)	Highest presence
Technical lead and architect	12–20	All phases
Service development (Java and .NET)	36–52	F3 and F4
Web application development (Angular)	14–20	F3 and F4
Mobile application development (React + Expo)	14–20	F3 and F4
Quality and testing	12–20	F4, with continuous support in F3
Operations and deployment	8–10	F2, F5, and F6
Interface and experience design	4–6	F2 and F3
Project management and acceptance	7–8	All phases
Total	**107–156**	

11.4 Adjustment scenarios

Table 38. Effort adjustment scenarios

Scenario	Estimated effort adjustment
Late confirmation of the GPS or notification provider	Increase of 8% to 15% in integration and testing
Extension of the scope to more than one institution in the first delivery	Increase of 10% to 20%
Reduction of the scope to the core epics (users, routes, QR, and notifications)	Reduction of 20% to 30%

Scenario	Estimated effort adjustment
Requirement for scaling with full orchestration in production	Increase of 5% to 10%

The percentages are planning references and do not constitute a contractual commitment.

12 Risk Analysis

12.1 Assessment criteria

IMPACT

│

│ HIGH │ \[Mitigate\] │ \[Avoid\] │

│ │ Probability │ Probability │

│ │ low │ high │

│──

─

│ MEDIUM │ \[Accept and │ \[Mitigate\] │

│ │ monitor\] │ │

│──

─

│ LOW │ \[Accept\] │ \[Accept\] │

└──

─

LOW HIGH

PROBABILITY

Figure 6. Probability and impact matrix

The available strategies are mitigate (reduce probability or impact), avoid (change the plan), transfer (shift the risk to a third party), and accept (record and monitor with a contingency plan).

12.2 Risk register

Table 39. Risk register and mitigations

ID	Risk	Category	Probability	Impact	Strategy	Mitigation
R-01	The GPS provider or its contract is not confirmed before live integration.	External	Medium	High	Mitigate	Start contract management in advance, develop with simulated data, and enable live tracking when the provider is confirmed.
R-02	The push notification provider is not confirmed for production.	External	Medium	High	Mitigate	Register the token per device and maintain a retryable delivery queue; offer the in-app notification as a contingency.

ID	Risk	Category	Probability	Impact	Strategy	Mitigation
R-03	Guardians do not use the application as often as expected.	Product	Medium	High	Mitigate	Run a pilot and measure consultation frequency; support adoption with institutional communicati on.
R-04	Drivers do not scan the QR correctly.	Operations	Medium	High	Mitigate	Test pickup and drop-off on real journeys, train staff, and offer immediate feedback in the application.
R-05	The GPS signal is unstable during the journeys.	Technical	Medium	High	Mitigate	Test the device on different journeys, show the last known position with a "no signal" marker, and manage reconnection.

ID	Risk	Category	Probability	Impact	Strategy	Mitigation
R-06	Notifications are not timely or turn out to be irrelevant.	Product	Medium	High	Mitigate	Measure generation and delivery times, parameterize the thresholds, and control the frequency of the notices.
R-07	The transportation process varies between institutions.	Product	Medium	Medium	Mitigate	Validate the flow with each institution before standardizing and keep configurable parameters.
R-08	The institution's initial data load is delayed or arrives incomplete.	Data	Medium	High	Mitigate	Perform the load in an assisted manner and validate the data before the acceptance tests.

ID	Risk	Category	Probability	Impact	Strategy	Mitigation
R-09	Users do not perceive value in the location consultation.	Product	Medium	Medium	Mitigate	Measure usage during the pilot and adjust the experience according to the evidence.
R-10	The retention of positioning data is not defined before launch.	Compliance	Medium	Medium	Accept and monitor	Define the retention policy with the legal area before go-live.
R-11	The scope grows with functions unrelated to the traceability objective.	Management	Medium	Medium	Avoid	Keep the agreed scope and channel new needs toward a future version.

12.3 Technical risks of the architecture

Table 40. Technical risks of the architecture

Technical risk	Typical probability	Mitigation
Cascade of failures due to the fall of a service	Medium	Circuit breakers, timeouts, fallback responses, and a 503 response when a critical dependency is down.
Data inconsistency between services	High	Idempotent consumers, events with ordering by key, and compensation patterns in multi-service assignments.

Technical risk	Typical probability	Mitigation
Latency in synchronous calls	High	Use of gRPC for telemetry, cache at the edge, and asynchronous processing where possible.
Message loss in the broker	Medium	At-least-once delivery with idempotent consumers and controlled retries.
Schema evolution that breaks consumers	High	Additive versioning of events and contract tests before release.
Accumulation of technical debt	High	Definition of done with minimum coverage, periodic reviews, and a prioritized debt register.
Exposure of sensitive data in logs	Medium	Logging policy, masking of personal data, and static security analysis.
Configuration drift between environments	High	Infrastructure as code, environment variables, and templates verified by the pipeline.
Premature over-engineering	Medium	Build simply, measure before optimizing, and prioritize the reliability of the core flow.

12.4 Open questions that condition decisions

Table 41. Open questions that condition decisions

ID	Question	Owner	Necessary before
Q-01	How many days is positioning data retained?	Product and legal area	Go-live
Q-02	What reports with personal data may leave the administrative roles?	Security	Go-live
Q-03	What are the recovery point and time objectives per schema?	Operations	Go-live

ID	Question	Owner	Necessary before
Q-04	What offline tolerance must the estimated time of arrival calculation support?	Domain	Next increment

13 Success Criteria

13.1 Key results

Table 42. Key results and measurement methods

KR	Indicator	Target	Measurement method
KR1	Platform availability	Monthly SLO of 99.9%	Health checks and gateway metrics
KR2	API latency at critical points	P95 < 300 ms	Load tests with k6 in the testing environment before release
KR3	Coverage of monitored routes	At least 95% of active routes with available position	Positioning data received during execution
KR4	Rate of delivered notifications	At least 95% of relevant events generate and deliver their notification	Event and delivery logs
KR5	Boarding recording without incidents	At least 95% of pickups and drop-offs recorded by QR without incidents	Boarding records generated by the system
KR6	Adoption by role	Monthly active users by role, with a target to be set during the pilot	Authentication and activity metrics by role

13.2 Complementary indicators

Table 43. Complementary success indicators

Indicator	Target	Observation
Transportation information managed on the platform	At least 90% of operational records	Replaces manual and scattered processes.
North-star metric: journeys with critical events recorded and available	Growing coverage per journey	Integrates bus position and boardings available to the corresponding users.
End-to-end timeliness	From the boarding event to the guardian's notification in less than 5 seconds (P95)	Operational objective of the notification flow.
Reduction of information-unavailable incidents	Reduction against the pilot baseline	Measured with the support and incident logs.

13.3 Acceptance Criteria of the implementation

The system is considered accepted when the test suite in the testing environment is passed —unit, integration, end-to-end, contract, load, and security—, the availability objective and the P95 latency are verified, the smoke tests are satisfactory in each environment, the security review reports no pending critical findings, and the institution formally approves the delivered scope.

14 Conclusions and next steps

14.1 Conclusions

The proposal solves a concrete and verifiable problem: the absence of a single, timely source of information about school transportation. The combination of GPS tracking, QR recording of pickup and drop-off, automatic notifications, an administrative panel, and mobile applications turns a manual and fragmented process into a traceable, auditable process centered

on student safety.

The chosen architecture —event-driven microservices, a single gateway, schema per service, and web and mobile clients— offers the right balance between delivery speed, data integrity, and growth capacity. Security decisions are incorporated from the design stage: authentication with an asymmetric signature, role-based authorization with deny by default, auditing of critical actions, and responsible handling of minors' data.

The phased plan and the proposed schedule make it possible to deliver value progressively. The verifiable milestones —base environment, accounts and routes, trip operation, guardian visibility, and go-live— give the institution clear control points. The effort estimate is presented as a range and adjusts when the external dependencies are confirmed, in particular the GPS and notification providers.

14.2 Next steps

1. Approve the proposed scope, objectives, and schedule, with the definition of the route and exception taxonomy.
2. Start managing the contracts with the GPS provider and the notification provider, in advance of the construction phase.
3. Close the open questions about positioning data retention, reports with personal data, and recovery objectives.
4. Agree with the institution on the initial data load plan and the acceptance criteria.

5. Form the team with the estimated dedications and enable the development and testing environments.

6. Start the analysis and validation phase, with progress review at the close of each milestone.